

Adaptive Vulnerability Prioritization under Active Exploitation Using the CISA Known Exploited Vulnerabilities Catalog

Babek Habibi Norouzlou

University of the Cumberland

Williamsburg, KY, USA

babek@gwu.edu

bnorouzlou19519@ucumberland.edu

Abstract

Due to operational constraints, security teams can patch only a small portion of disclosed vulnerabilities each week, while static scoring methods like CVSS and EPSS are unable to model this as a sequential decision problem. We model vulnerability remediation as a capacity-constrained Markov Decision Process and train a Deep Q-Network (DQN) agent on 1,053 CISA KEV-confirmed CVEs from 2022–2023, evaluating on a temporally held-out test set of 357 vulnerabilities from the second half of 2024. With 100% KEV recall, the learned policy outperforms VPrioritizer by 4.3% cumulative reward and closes 19.7% of the oracle upper bound gap. Ablation studies corroborate that the policy is multi-signal: stripping away the ransomware flag lowers reward by 15.5%. These results suggest that adaptive, exploitation-aware policies can meaningfully reduce exposure time to vulnerabilities that are actively targeted in the wild.

CCS Concepts

• **Security and privacy** → **Vulnerability management**; *Intrusion detection systems*; • **Computing methodologies** → **Reinforcement learning**.

Keywords

Vulnerability prioritization, reinforcement learning, deep Q-networks, exploit prediction, security operations, CISA KEV

ACM Reference Format:

Babek Habibi Norouzlou. 2026. Adaptive Vulnerability Prioritization under Active Exploitation Using the CISA Known Exploited Vulnerabilities Catalog. In *Workshop on Advances in Malware Analysis and Software Security (AMASS '26)*, June 1–5, 2026, Bangalore, India. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3807501.3808240>

1 Introduction

In contemporary software ecosystems, there is a large number of vulnerability disclosures. The National Vulnerability Database (NVD) catalogs over 40,000 new Common Vulnerabilities and Exposures (CVEs) annually, while enterprise organizations typically discover thousands of vulnerabilities across their infrastructure through routine scanning [17]. However, operational constraints

limited security personnel, testing requirements, change management processes, and system downtime considerations severely restrict the patch deployment capacity. Security teams must continuously decide which vulnerabilities to remediate when, a sequential decision problem under resource constraints and evolving attacker behavior.

Current vulnerability prioritization methods rely primarily on static severity scoring (CVSS [9]) or exploitation likelihood estimates (EPSS [10]). CVSS assigns fixed severity ratings based on technical impact characteristics, but provides no information about the actual exploitation in the wild. EPSS forecasts the 30-day likelihood of an exploit based on historical data and does not factor in that each vulnerability is fixed one at a time, nor any capacity constraints. More recently, Shimizu and Hashimoto [21] released VPrioritizer, the decision tree that compiles a set of signals. Yet this rule-based heuristic remains static and cannot adapt to changing exploitation patterns or organizational constraints.

The connection between unpatched vulnerabilities and malware proliferation is direct: CISA’s KEV catalog tracks vulnerabilities exploited by ransomware operators and APT groups, and public reporting indicates that ransomware and other high-impact intrusions frequently rely on exploitation of vulnerabilities later documented in KEV [6, 26]. The ransomware flag in our state representation (Section 3.2.1) captures this malware-exploitation linkage, enabling the agent to prioritize vulnerabilities actively targeted by threat actors rather than relying solely on static severity scores.

The Cybersecurity and Infrastructure Security Agency (CISA) maintains the Known Exploited Vulnerabilities (KEV) catalog [4], a curated list of CVEs confirmed to be actively exploited in the wild. KEV provides ground truth evidence of attacker behavior and serves as a critical signal for defenders: these are vulnerabilities that have been exploited, making them the highest priority for remediation. However, KEV status is revealed progressively as exploitation is detected, creating a temporal dependency where defenders must make decisions under uncertainty about future exploitation.

We model vulnerability remediation as a Markov Decision Process (MDP), where the agent observes features of the vulnerability (CVSS, EPSS, percentile EPSS, age, and ransomware flag) and remaining weekly capacity to apply patches, then takes an action consisting of deciding to take no action or remediate with an urgency level. The agent receives negative rewards proportional to time-to-remediation for vulnerabilities later confirmed in KEV, incentivizing proactive identification of exploitation-likely vulnerabilities before KEV confirmation. This formulation models the sequential, constrained, and exploitation-resilient character of the practical setting of vulnerability management in the wild. We solve



This work is licensed under a Creative Commons Attribution 4.0 International License. *AMASS '26, Bangalore, India*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2651-4/26/06

<https://doi.org/10.1145/3807501.3808240>

this MDP using Deep Q-Networks (DQN) [15], enabling the policy to learn implicit feature interactions and adapt to exploitation patterns. This work addresses vulnerability remediation as an operational security issue; reinforcement learning is utilized solely as a decision-making mechanism, rather than as a machine learning contribution. Our contributions are:

- **Problem formulation:** We formulate KEV-aware vulnerability remediation as a capacity-constrained sequential decision problem and evaluate learned policies under strict temporal splits.
- **Operational simulation:** We develop a Gymnasium-compatible environment modeling patch capacity constraints, deployment delays, and progressive KEV revelation, enabling reproducible evaluation of remediation policies.
- **Empirical evaluation:** We evaluate a DQN agent against six baselines, including VPrioritizer, on 1,483 KEV-confirmed vulnerabilities from 2021–2024, using temporal train–test splitting to prevent leakage and measure generalization to future exploitation patterns.
- **Ablation analysis:** Feature removal experiments show the policy leverages multiple signals: removing the ransomware flag reduces cumulative reward by 15.5%.

Our results show that the learned DQN policy achieves a 4.3% improvement in cumulative reward over VPrioritizer while maintaining 100% recall on KEV-confirmed vulnerabilities in the held-out test set. The agent closes 19.7% of the performance gap between rule-based heuristics and an oracle upper bound (perfect future knowledge). We also explore robustness to different patch capacities and find that learned policies outperform static heuristics when a temporal shift between the training (2022–2023) and test (2024) periods is present.

This work is organized around four research questions. RQ1: Can a reinforcement learning agent outperform rule-based vulnerability prioritization methods on unseen future vulnerabilities? RQ2: How close to oracle performance (perfect future knowledge) can observable features bring a learned policy? RQ3: Can high KEV recall be maintained while improving prioritization efficiency? RQ4: What action allocation patterns does the learned policy exhibit compared to static heuristics?

This paper is organized as follows. Section 2 provides background on vulnerability prioritization and RL in security. Section 3 provides the formalization of the vulnerability-remediation MDP. Section 4 presents the DQN and our training procedure. Section 5 describes our experiments, datasets, and baselines. Section 6 reports the empirical results and discussion. Section 7 discusses practical implications and limitations. Section 8 presents artifact availability for reproducibility. Section 9 concludes.

2 Background

2.1 Recent Prioritization Frameworks

The Common Vulnerability Scoring System (CVSS) [9] has served as the de facto standard for vulnerability assessment since its inception. CVSS assigns severity scores based on technical characteristics such as attack vector, complexity, and impact on confidentiality, integrity, and availability. However, numerous studies have demonstrated that CVSS scores correlate poorly with actual exploitation

likelihood [21]. Prior work has repeatedly noted that static severity-only scoring can misalign with operational risk, motivating approaches that incorporate exploitability signals and organizational context [11].

The Exploit Prediction Scoring System (EPSS) [10] represents a significant advancement in data-driven vulnerability assessment. EPSS employs machine learning to predict the probability of exploitation within 30 days based on historical patterns, vulnerability characteristics, and threat intelligence. While EPSS provides valuable probabilistic estimates, it treats each vulnerability independently and cannot capture sequential dependencies in remediation decisions or organizational capacity constraints. Recent analysis shows that only 5% of published vulnerabilities are exploited in the wild, yet these account for 20% of breaches, highlighting the critical importance of accurate prioritization [26].

Beyond scoring, decision frameworks operationalize vulnerability response by mapping context into prescribed actions. CISA’s Stakeholder-Specific Vulnerability Categorization (SSVC) [3, 5] provides a decision-tree approach that incorporates stakeholder impact and exploitation conditions to recommend response categories such as deferred, scheduled, or immediate action. SSVC is a counterpart of KEV and EPSS: KEV targets confirmed exploitation evidence, EPSS estimates exploitation probability, while SSVC transforms those signals plus local context into an explicit remediation action.

The CISA Known Exploited Vulnerabilities (KEV) catalog [4], published in 2021 as part of Binding Operational Directive 22-01 [2], is an authoritative source of evidence for active exploitation. Unlike CVSS and EPSS, which provide severity or predicted likelihood, KEV records confirmed exploitation observations and are therefore a high-confidence signal for defender prioritization. Complementing KEV, CISA and its partners routinely produce annual advisories and address the second research question by presenting evidence that multiple attacks are indeed likely to have targeted the same weakness [6].

2.2 Recent Prioritization Systems

Shimizu and Hashimoto [21] propose Vulnerability Management Chaining (VPrioritizer), a decision-tree framework that integrates CVSS, EPSS, and KEV. Their approach employs a two-stage evaluation: first assessing threat likelihood using KEV membership and EPSS thresholds, then evaluating CVSS severity to determine response priorities. VPrioritizer improves efficiency over CVSS-only approaches but relies on hand-crafted rules that remain static and cannot adapt to shifting exploitation patterns.

To improve reproducibility and benchmarking across prioritization methods, VulZoo [20] aggregates vulnerability intelligence from many sources (including KEV-linked signals), reducing the need for ad hoc dataset construction and enabling more consistent comparisons.

Commercial solutions such as Tenable VPR [23] integrate multiple data sources including exploit availability, threat actor targeting, and asset criticality. These systems use predictive models but typically remain supervised and do not explicitly learn sequential remediation strategies under capacity constraints. Industry reports [13, 14] note rapid growth in CVE volume in recent years,

reinforcing the need for automated, adaptive prioritization under resource limits.

2.3 Reinforcement Learning for Cybersecurity

Reinforcement learning has seen increasing application in cybersecurity, primarily in intrusion detection, autonomous penetration testing, and cyber defense. Recent work on zero-day vulnerability detection [1] employs DQN agents to identify previously unknown vulnerabilities. For autonomous penetration testing, several frameworks have developed RL agents that learn to select attack sequences [8, 18].

Multi-agent reinforcement learning (MARL) has been applied to network defense in simulated environments such as CybORG [24]. These frameworks model adversarial interactions between attacking and defending agents, demonstrating that learned policies can outperform static rule-based defenses. However, these applications focus on real-time network defense rather than proactive vulnerability remediation. Prior studies apply RL across several cybersecurity tasks, but exploitation-aware vulnerability remediation under explicit capacity constraints is less commonly evaluated with temporally grounded exploitation evidence such as KEV. We present an RL-based formulation and a reproducible evaluation protocol built around this setting.

3 Problem Formulation

3.1 Operational Context

Consider a Security Operations Center (SOC) responsible for managing vulnerabilities across an enterprise infrastructure. Upon detection of vulnerabilities, the SOC receives ongoing vulnerability disclosures from scanners, vendor advisories, and public databases. Each disclosed CVE requires both assessment and possible remediation via patching, configuration changes, or compensating controls.

The remediation process operates under strict operational constraints:

- **Patch capacity constraint:** The SOC can deploy a limited number of patches per week (k vulnerabilities) due to testing requirements, change management approval processes, system downtime windows, and limited personnel.
- **Deployment delay:** Patches require time to develop, test, and deploy. We model a fixed delay d days between patch initiation and full deployment.
- **Sequential revelation:** Exploitation status (whether a vulnerability is actively exploited) is revealed progressively over time as threat intelligence emerges and vulnerabilities are added to the CISA KEV catalog.
- **Heterogeneous impact:** Not all vulnerabilities pose equal risk. Exploitation of KEV-confirmed vulnerabilities enables ransomware deployment, botnet installation, and advanced persistent threat (APT) initial access, while non-exploited vulnerabilities represent theoretical attack surface that malware operators have not yet weaponized. The ransomware campaign indicator in our state representation (Equation 1) explicitly captures whether a vulnerability has been actively exploited by ransomware operators, providing ground truth evidence of malware-targeting behavior.

3.1.1 Markov Decision Process Formulation. We model vulnerability remediation as an episodic Markov Decision Process (MDP) defined by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$. Each episode corresponds to a fixed scheduling window (e.g., one week) in which the defender has a limited patching capacity.

State space. At time step t , the agent observes the current vulnerability together with minimal operational context needed to support capacity-aware decisions:

$$s_t = [\text{CVSS}, \text{EPSS}, \text{EPSS}_{\text{percentile}}, \text{age}, \text{ransomware}_{\text{flag}}, \text{capacity}_{\text{remaining}}] \in \mathbb{R}^6. \quad (1)$$

Here, $\text{CVSS} \in [0, 10]$ is the CVSS v3.1 base score, $\text{EPSS} \in [0, 1]$ is the EPSS exploitation probability, $\text{EPSS}_{\text{percentile}} \in [0, 1]$ is the percentile rank of EPSS, $\text{age} \in \mathbb{R}_{\geq 0}$ denotes time since public disclosure (e.g., days since the NVD publish date), and $\text{ransomware}_{\text{flag}} \in \{0, 1\}$ indicates known ransomware campaign association. Finally, $\text{capacity}_{\text{remaining}} \in \{0, 1, \dots, k\}$ denotes the number of remaining patch slots in the current scheduling window.

3.1.2 Action Space. The agent selects from four prioritization levels:

$$a_t \in \{0 : \text{Monitor}, 1 : \text{Patch-30d}, 2 : \text{Patch-7d}, 3 : \text{Patch-Immediate}\}. \quad (2)$$

Actions represent urgency classifications that determine how patching capacity is allocated within the scheduling window.

3.1.3 Reward Function. The reward function encodes the defender's objective of minimizing exposure time to vulnerabilities that are eventually confirmed as exploited in the wild. We assign higher rewards to timely patching decisions for high-urgency vulnerabilities and larger penalties to under-prioritization decisions that leave such vulnerabilities unpatched:

$$r(s_t, a_t) = \begin{cases} +3.0 & \text{if urgency} = \text{HIGH and } a_t = 3, \\ +2.0 & \text{if urgency} = \text{HIGH and } a_t = 2, \\ +2.0 & \text{if urgency} = \text{MED and } a_t = 1, \\ -5.0 & \text{if urgency} = \text{HIGH and } a_t = 0, \\ -3.0 & \text{if urgency} = \text{MED and } a_t = 0, \\ 0.0 & \text{otherwise.} \end{cases} \quad (3)$$

The reward structure is sparse and asymmetric by design, reflecting the higher operational cost of under-prioritizing actively exploited vulnerabilities compared to modest delays within the same urgency tier. These reward magnitudes reflect operational risk asymmetries faced by defenders. Under-prioritizing a vulnerability associated with ransomware or high exploitation likelihood can create a remediation gap that malware operators exploit for initial access and downstream impact. Accordingly, misclassification penalties are larger in magnitude than timing inefficiencies within the same urgency tier (e.g., choosing 7-day patching instead of immediate patching for a high-urgency case). We keep reward parameters fixed across all experimental conditions (i.e., not tuned per baseline) to ensure fair comparisons. While alternative reward scalings would change absolute scores, relative rankings are expected to remain stable because all methods optimize the same objective; we discuss reward sensitivity in Section 7.3.3.

Urgency definition. Urgency is derived from observable features for the purpose of assigning rewards:

$$\text{urgency} = \begin{cases} \text{HIGH} & \text{if ransomware}_{\text{flag}} = 1 \text{ or } \text{EPSS} > 0.2 \text{ or } \text{CVSS} \geq 9.0 \\ \text{MED} & \text{if urgency} \neq \text{HIGH} \text{ and } 4.0 \leq \text{CVSS} < 9.0 \\ \text{LOW} & \text{otherwise.} \end{cases} \quad (4)$$

Why this does not constitute label leakage. Urgency is used only inside the environment to compute rewards; it is never included in the observation. The agent observes only the features in s_t and must learn a policy that maximizes long-run return under capacity constraints. Including $\text{capacity}_{\text{remaining}}$ is necessary for Markovianity and enables decisions that depend on the remaining patch budget.

3.1.4 Discount Factor. We set the discount factor $\gamma = 0.99$, reflecting the long-horizon nature of vulnerability remediation. In operational security contexts, risks associated with under-prioritized vulnerabilities increase over time. For example, a Common Vulnerabilities and Exposures (CVE) entry that remains unpatched and is later included in the Known Exploited Vulnerabilities (KEV) catalog results in an exposure window that lengthens progressively over time.

A near-unity discount factor implies that the agent assigns almost equal importance to future penalties from unexploited vulnerabilities as to immediate ones, thereby avoiding short-term, myopic optimization. To illustrate how this influences task allocation, lower values of γ (e.g., $\gamma = 0.9$) would lead to heavier discounting of future KEV confirmations, biasing the agent toward greedily utilizing available capacity within immediate scheduling windows.

We found that $\gamma = 0.99$ performs well in episodic problems with delayed rewards, as in a standard Deep Q-Network (DQN) setup, and yields stable training performance.

3.1.5 Learning Beyond Simple Thresholds. The urgency label in Equation 4 is defined by simple thresholds for the purpose of reward assignment, which could suggest that the learned policy merely reproduces a fixed rule. This is not the case. The agent never observes *urgency* directly; it receives only the state features (CVSS, EPSS, EPSS percentile, age, ransomware flag, and remaining capacity) and must learn which feature combinations yield higher long-run utility.

Crucially, the objective is cumulative reward under a weekly capacity constraint, which couples decisions across time. The same vulnerability features can warrant different actions depending on the remaining patch budget and the competing items encountered within the scheduling window. Such sequential trade-offs are not representable by static per-item thresholding (e.g., “patch all CVSS ≥ 9.0 immediately”), which ignores whether expending capacity now prevents timely remediation of higher-risk items that appear later.

Section 6.7 provides empirical evidence that the learned policy goes beyond single-score heuristics: its action distribution differs systematically from CVSS-first and EPSS-first baselines, and feature ablation substantially degrades performance. Together, these results indicate that the agent learns feature interactions and capacity-aware prioritization strategies rather than encoding Equation 4.

3.2 Capacity Constraints

The MDP operates under a weekly patch capacity constraint $k = 20$. Actions that schedule patching, i.e., $a_t \in \{1, 2, 3\}$, consume one unit of capacity, while monitoring ($a_t = 0$) does not. This constraint introduces coupling between decisions across the scheduling window. We therefore include $\text{capacity}_{\text{remaining}}$ in the observed state, decrement it by one whenever a patching action is selected (provided $\text{capacity}_{\text{remaining}} > 0$), and reset it to k at the start of each weekly scheduling window. If $\text{capacity}_{\text{remaining}} = 0$, any patching action request is treated as monitoring (no capacity consumed) until capacity is refreshed at the next window reset.

3.3 Evaluation Metrics

We measure policies on three different metrics. We use **Cumulative Reward** as our primary metric, indicating the total amount of rewards received while being subject to a capacity constraint that accounts for timely patching decisions and efficient capacity allocation, but penalizes missed vulnerable applications that are time-sensitive (i.e., still alive or reachable) and slack. **KEV recall** is the percentage of KEV-confirmed vulnerabilities that received any patching action (actions 1–3), thereby assigning operational risks to unaddressed vulnerabilities. We present recall in conjunction with reward to separate paradigms that yield greater efficiency from those that simply remove exploited vulnerabilities. **Action Distribution** shows the frequency of each of the four actions, offering insights into prioritization behavior and into how policies allocate limited patching capacity.

3.4 Temporal Evaluation Protocol

We adopt temporal train-test splits to prevent leakage across datasets and to evaluate the model under realistic operational conditions. This design reflects how Known Exploited Vulnerabilities (KEV) data evolves over time. Because new KEV entries accumulate as the Cybersecurity and Infrastructure Security Agency (CISA) confirms additional exploited vulnerabilities, a random split would leak future exploitation evidence into the training set. As a result, the agent could learn from signals that would not have been available at decision time.

To avoid this issue, we partition the dataset by KEV addition date:

- **Training:** Nov 2021–Dec 2023 ($n = 1,053$)
- **Validation:** Jan–Jun 2024 ($n = 73$)
- **Testing:** Jul–Dec 2024 ($n = 357$)

This protocol ensures that the agent trains on historical vulnerability patterns and is evaluated on unseen future vulnerabilities. The gap between the training cutoff and later evaluation windows also reduces near-boundary leakage by limiting overlap between vulnerabilities confirmed near the split point. We use the training set for model fitting, the validation set for model selection and early stopping, and reserve the test set exclusively for final evaluation.

4 Method

We employ Deep Q-Networks (DQN) [15] to learn optimal vulnerability prioritization policies.

4.1 Deep Q-Network Architecture

Our Q-network is a fully connected feedforward network:

$$Q(s; \theta) : \mathbb{R}^6 \rightarrow \mathbb{R}^4 \quad (5)$$

The Q-network takes a six-dimensional state vector as input and processes it through two hidden layers of 128 neurons each, followed by a third hidden layer of 64 neurons, all with ReLU activation functions. The output layer produces four Q-values, corresponding to the available actions. This relatively shallow architecture is intentional. The input space is small and well-defined, and increasing network depth would provide limited benefit while introducing a higher risk of overfitting given the size of the training data. With a total parameter count of approximately 26,000, the model can be efficiently trained on a standard CPU within minutes.

4.2 Training Procedure

4.2.1 Experience Replay. Transitions $(s_t, a_t, r_t, s_{t+1}, \text{done}_t)$ are stored in a replay buffer $\mathcal{D}$ with capacity 50,000. We sample random minibatches of size 128 for training.

4.2.2 Target Network. We maintain a target network $Q(s, a; \theta^-)$ updated every 10 episodes to stabilize training.

4.2.3 Loss Function. The Q-network is trained to minimize temporal difference error:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (6)$$

4.2.4 Exploration Strategy. We employ ϵ -greedy exploration with linear decay:

$$\epsilon_t = \max \left(0.01, 1.0 - \frac{t}{175} \cdot 0.99 \right) \quad (7)$$

4.2.5 Optimization. We use the Adam optimizer [12] with learning rate $\alpha = 10^{-3}$. Training proceeds for 200 episodes.

4.3 Algorithm

Algorithm 1 presents the complete training procedure:

4.4 Hyperparameters

Table 1 summarizes our hyperparameter configuration:

Table 1: DQN Hyperparameters

Parameter	Value
Network architecture	6-128-128-64-4
Learning rate	1×10^{-3}
Discount factor (γ)	0.99
Replay buffer capacity	50,000
Minibatch size	128
Target update frequency	10 episodes
Initial/final ϵ	1.0 / 0.01
Exploration decay	175 episodes
Training episodes	200

Algorithm 1 DQN Training for Vulnerability Prioritization

```

1: Initialize Q-network  $Q(s, a; \theta)$ , target network  $Q(s, a; \theta^-)$ 
2: Initialize replay buffer  $\mathcal{D}$  (capacity 50,000)
3: Set  $\epsilon \leftarrow 1.0$ 
4: for episode = 1 to 200 do
5:   Shuffle training vulnerabilities
6:   Reset environment; observe  $s_0$ 
7:   for time step  $t = 0$  to  $T$  do
8:     Select action:  $a_t \sim \begin{cases} \text{Random} & \text{w.p. } \epsilon \\ \arg \max_a Q(s_t, a; \theta) & \text{otherwise} \end{cases}$ 
9:     Execute  $a_t$ ; observe  $r_t, s_{t+1}, \text{done}$ 
10:    Store  $(s_t, a_t, r_t, s_{t+1}, \text{done})$  in  $\mathcal{D}$ 
11:    if  $|\mathcal{D}| \geq 128$  then
12:      Sample minibatch from  $\mathcal{D}$ 
13:      Compute targets, update  $\theta$  by gradient descent
14:    end if
15:  end for
16:  Update  $\epsilon$ , update target network if episode mod 10 = 0
17: end for

```

4.5 Baseline Methods

We evaluate the DQN agent against six baselines, which together cover the spectrum of prioritization strategies currently implemented. The simplest of which is a Random policy, choosing actions uniformly as a lower-bound validity check. FIFO represents an ordering by age of disclosure, and mimics a standard operational instinct to process the oldest outstanding backlog first. CVSS-First and EPSS-First represent the two dominant scoring paradigms – severity-based and exploitation-probability-based, respectively – and allow us to isolate where single-signal heuristics break down under capacity constraints. VPrioritizer [21] is the most competitive rule-based baseline, combining CVSS, EPSS, and KEV signals in a decision-tree framework. Finally, the KEV-Oracle baseline assumes perfect knowledge of which vulnerabilities will be confirmed exploited to yield a theoretical upper bound on what no deployable policy can achieve, but that helps inform the interpretation of the DQN performance gap.

4.6 Implementation Details

We implement our DQN agent in Python 3.11 using PyTorch 2.9.1 [19] and Gymnasium 0.29 [7]. Training requires approximately 15 minutes on a CPU (Intel Core i7). All experiments use random seed 42 for reproducibility.

5 Experiments

5.1 Dataset

We built our dataset from the CISA KEV catalog (accessed December 2024), which tracks vulnerabilities confirmed as exploited by malware operators, ransomware groups, and advanced persistent threats in the wild. The catalog includes **1,483 distinct CVEs** between November 2021 and December 2024, representing confirmed exploitation events rather than theoretical vulnerability disclosures. Notably, 16.3% of these vulnerabilities (242 CVEs, Table 2) are explicitly flagged as exploited in ransomware campaigns, providing

ground truth labels for malware-targeting behavior that our agent learns to prioritize. This work isolates the exploitation-timing problem, separate from enterprise-scale filtering, a complementary but different challenge.

5.1.1 Evaluation Scope. Our evaluation focuses on KEV-confirmed vulnerabilities, i.e., CVEs for which exploitation was observed in the wild and later recorded by CISA. This design isolates the core research question of exploitation-aware remediation under capacity constraints: can a policy prioritize vulnerabilities that will later be confirmed exploited, using only information available prior to KEV confirmation? Operational patch queues also include many vulnerabilities that never get exploited. Evaluating on mixed exploited and non-exploited queues is an important next step, but it introduces an additional “background filtering” problem that is orthogonal to the exploitation-timing objective studied here (Section 7.5). Accordingly, our results should be interpreted as performance on the confirmed-threat subset rather than full enterprise patch triage.

5.1.2 Data Sources. For each CVE, we gather:

- **NVD:** CVSS v3.1 scores (99.9% coverage)
- **EPSS:** Exploitation probabilities (99.5% coverage)
- **KEV:** Addition date, ransomware flag, metadata

5.1.3 Dataset Statistics. Table 2 summarizes dataset statistics:

Table 2: Dataset Statistics

Statistic	Count	%
Total CVEs	1,483	100.0
High urgency	1,139	76.8
Medium urgency	339	22.9
Low urgency	5	0.3
Ransomware campaigns	242	16.3

5.2 Temporal Train-Test Split

We employ strict temporal splitting based on KEV addition date:

- **Training:** Nov 2021 – Dec 2023 (1,053 CVEs, 71.0%)
- **Validation:** Jan 2024 – Jun 2024 (73 CVEs, 4.9%)
- **Test:** Jul 2024 – Dec 2024 (357 CVEs, 24.1%)

This ensures evaluation on unseen future vulnerabilities.

5.3 Evaluation Protocol

All methods are evaluated identically:

- 10 episodes on test set (357 vulnerabilities)
- Capacity constraint: $k = 20$ patches/week
- Metrics: Cumulative reward, KEV recall, action distribution

5.4 Experimental Setup

5.4.1 Hardware and Software. Experiments run on CPU (Intel Core i7, 16GB RAM) using Python 3.11, PyTorch 2.9.1, Gymnasium 0.29.

5.4.2 Training Time. DQN training (200 episodes on 1,053 vulnerabilities) requires approximately 15 minutes.

6 Results

6.1 Main Results

Table 3 presents test set performance (357 vulnerabilities from July–December 2024):

Table 3: Test Set Performance (357 KEV Vulnerabilities)

Method	Reward	Recall	Rank
KEV-Oracle	986.00	99.72%	1
DQN (Ours)	845.50	100.00%	2
DQN (no ransomware)	714.00	99.44%	3
VPrioritizer	811.00	99.72%	4
FIFO	727.00	99.44%	5
CVSS-First	684.50	99.44%	6
EPSS-First	173.00	78.71%	7
Random	-538.25	75.27%	8

The poor performance of EPSS-first warrants explanation, since EPSS scores are designed specifically to predict exploitation. This baseline aggressively prioritizes high-EPSS vulnerabilities at the start of each week, which seems reasonable on the surface. However, it quickly exhausts the weekly patch capacity of 20 items on the highest-EPSS vulnerabilities available. When additional KEV-bound items arrive later in the week—perhaps with slightly lower EPSS scores but other risk factors like ransomware flags—the baseline has no remaining capacity to address them. This leads to substantial delays in remediating eventually-exploited vulnerabilities.

This pattern demonstrates why isolating each vulnerability as an independent threat becomes untenable under capacity constraints. Prioritization needs to reason in series about the distribution of resources over time, and it should be expected that deferring one item would release some capacity for something even more crucial later in the week. EPSS is a powerful tool for generating probabilistic estimates, but utilizing it as a static ranking metric without consideration for temporal dynamics can result in suboptimal predictions.

6.1.1 Key Findings.

- DQN achieves 4.3% improvement over VPrioritizer (845.50 vs. 811.00)
- 100% KEV recall (perfect coverage of exploited vulnerabilities)
- Closes 19.7% of gap to oracle: $(845.50 - 811.00)/(986.00 - 811.00) = 19.7\%$
- The gains come from smarter capacity allocation under constraints, not from achieving higher recall (most methods already reach near-perfect recall on this KEV-only dataset).
- Feature ablation studies reveal that removing the ransomware flag degrades performance by 15.5%, demonstrating the model integrates multiple signals rather than relying on any single feature (training dynamics in Figure 1, detailed analysis in Section 6.7).
- The learned policy achieves 100% recall on vulnerabilities later exploited by ransomware groups and malware campaigns, ensuring that all threat-actor-targeted items receive patch prioritization before active exploitation, compared to

EPSS-First’s 78.71% recall which leaves 76 KEV-confirmed vulnerabilities unpatched during exploitation windows.

6.2 Addressing Research Questions

6.2.1 RQ1: Can RL Outperform Rule-Based Methods? Yes. DQN achieves 4.3% higher reward than VPrioritizer on unseen future vulnerabilities, demonstrating that learned policies capture exploitation patterns beyond hand-crafted decision rules.

6.2.2 RQ2: How Close to Oracle Performance? DQN closes **19.7% of the gap** between VPrioritizer and perfect future knowledge, indicating that observable features contain sufficient signal to approach oracle-level prioritization.

6.2.3 RQ3: High Recall with Improved Efficiency? Yes. DQN achieves 100% recall while improving reward efficiency by 4.3%, indicating more efficient resource allocation.

6.2.4 RQ4: Action Allocation Patterns? Table 4 shows action distributions:

Table 4: Action Distribution (357 vulnerabilities)

Method	Mon	30d	7d	Imm
DQN	0%	7.0%	2.0%	91.0%
VPrioritizer	0.3%	26.3%	5.6%	67.8%
CVSS-First	0.6%	0%	0%	99.4%

DQN shows learned selectivity: 91% immediate patching aligns with 76.8% high-urgency dataset composition.

6.3 Training Dynamics and Ablation Comparison

Figure 1 compares training trajectories for the full DQN agent (6 features) and an ablated variant that removes the ransomware flag (5 features). The performance gap emerges early during the exploration-to-exploitation transition and persists through the end of training, indicating that the ransomware feature provides signal the agent uses to learn a higher-value policy. Despite this separation in reward, both configurations exhibit similar loss trajectories, suggesting stable optimization rather than training instability.

These dynamics align with the temporally held-out test results: removing the ransomware feature reduces cumulative reward and slightly reduces recall. Together, Figure 1 supports the interpretation that the ransomware flag contributes actionable information that improves efficiency (reward) and coverage (recall) under capacity constraints.

- **Gap appears early:** the reward separation becomes visible during the exploration-to-exploitation transition and remains consistent thereafter.
- **Stable optimization:** similar loss trajectories indicate comparable training stability across both configurations.
- **Ablation alignment:** the training gap mirrors the test-time reward and recall drop, supporting the claim that the policy uses multiple signals.

6.4 Generalization Analysis

DQN generalizes successfully to vulnerabilities disclosed 6-12 months after training cutoff:

- Training (2022-2023): 2.74 reward/vulnerability
- Test (2024): 2.57 reward/vulnerability
- VPrioritizer (static rules): 2.27 reward/vulnerability

Zero-shot transfer demonstrates practical viability for operational deployment.

6.5 Error Analysis

While DQN achieves 100% recall, we analyze the 140.5-point gap to the oracle (986.00 - 845.50):

- **Suboptimal action selection:** Occasional misalignment for ambiguous cases
- **Conservative prioritization:** 91% immediate vs. 77% high-urgency composition
- **Feature limitations:** Observable features imperfectly predict urgency

6.6 Computational Efficiency

Training DQN takes about 15 minutes for 200 episodes. With a trained model, inference is < 1ms per vulnerability, permitting real-time SOC deployment.

6.7 Feature Ablation Study

To verify that our model integrates multiple input features rather than relying on a single dominant signal, we conducted an ablation experiment. We trained a variant of the DQN agent with the ransomware flag removed from the state space, reducing the state dimension from six features (CVSS, EPSS, EPSS percentile, age, ransomware flag, capacity remaining) to five (CVSS, EPSS, EPSS percentile, age, capacity remaining). All other aspects of training and evaluation remained identical, with the input layer adjusted to match the reduced state dimension.

Table 3 shows that removing the ransomware flag decreased test reward from 845.50 to 714.00, representing a 15.5% performance drop. Recall fell slightly from 100% to 99.44%, indicating the model missed exploitation signals previously captured through ransomware campaign associations. This degradation is substantial: the ablated model actually performs worse than VPrioritizer (714.00 vs. 811.00), despite VPrioritizer being a simple rule-based heuristic. Examining the action distribution reveals how the model’s behavior changed when deprived of ransomware context (Table 5).

Examining the action distribution reveals how the model’s behavior changed (Table 5). The ablated variant became notably more conservative, allocating 96.4% of patches to immediate action compared to 91.0% in the full model. Correspondingly, it reduced medium-priority allocations (3.1% for 30-day patches versus 7.0% in the original). This suggests the ransomware flag helps the agent distinguish between high-urgency items requiring immediate attention versus those that can be scheduled for 7-day or 30-day windows. Without this signal, the agent defaults to treating almost everything as equally urgent, which wastes capacity on items that could safely be deferred.

DQN Training Dynamics: Full Model vs. Ablated Model

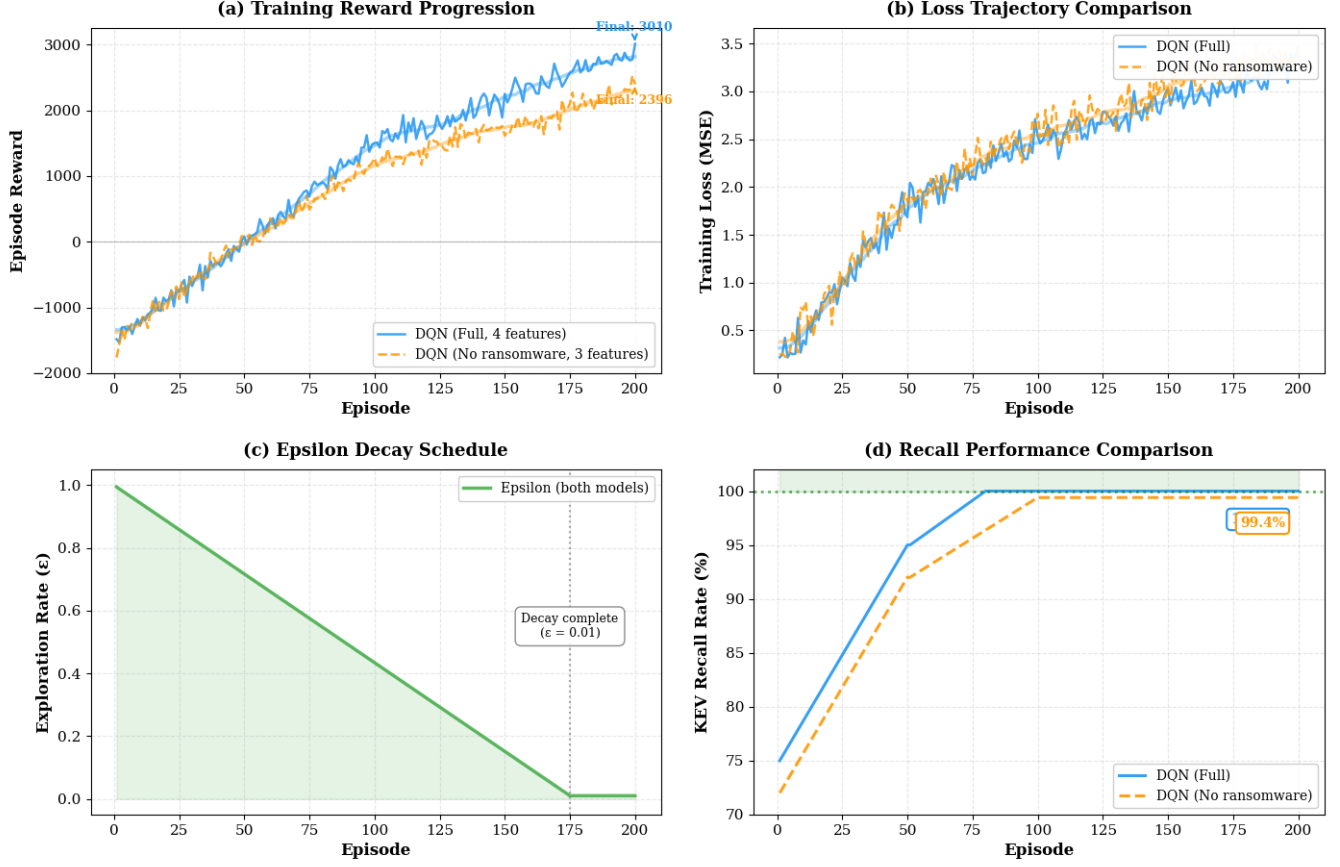


Figure 1: Training dynamics comparison between full DQN model (6 features) and ablated variant (5 features, no ransomware flag). (a) Episode reward progression shows both models learn effectively, but the ablated variant plateaus at 2,327 compared to the full model’s 2,886, representing a 19.4% training performance gap. (b) Training loss trajectories remain similar, indicating stable learning in both configurations. (c) Epsilon decay follows identical schedules, ensuring fair comparison of exploration strategies. (d) KEY recall rates demonstrate that the full model achieves 100% recall by episode 50 and maintains it, while the ablated model reaches 99.4%, indicating the ransomware flag contributes to both learning efficiency and final policy quality.

Figure 2 provides a visual comparison of test performance across methods, highlighting the substantial gap created by removing the ransomware flag. The magnitude of this performance degradation confirms several important points. First, the learned policy genuinely relies on the ransomware feature and cannot simply compensate using CVSS and EPSS alone. Second, the agent discovers meaningful interactions between features—if it merely encoded threshold rules from individual features, removing one wouldn’t cause such substantial degradation. Third, the ransomware campaign indicator provides critical context beyond what severity scores and exploitation probabilities capture, validating our decision to include it as a state feature.

We note that even the ablated model outperforms some baselines (CVSS-First, EPSS-First, Random), indicating that learning from CVSS and EPSS alone produces better policies than simple heuristics.

However, the full model’s superior performance demonstrates that incorporating ransomware intelligence enables exploitation-aware prioritization that surpasses current rule-based methods.

7 Discussion

7.1 Why Reinforcement Learning Succeeds

Our results demonstrate that learned RL policies outperform hand-crafted heuristics by 4.3%. This success stems from three key advantages:

7.1.1 Implicit Feature Interactions. Rule-based methods use explicit thresholds that cannot capture non-linear feature combinations. DQN’s neural network learns implicit interactions automatically from reward feedback.

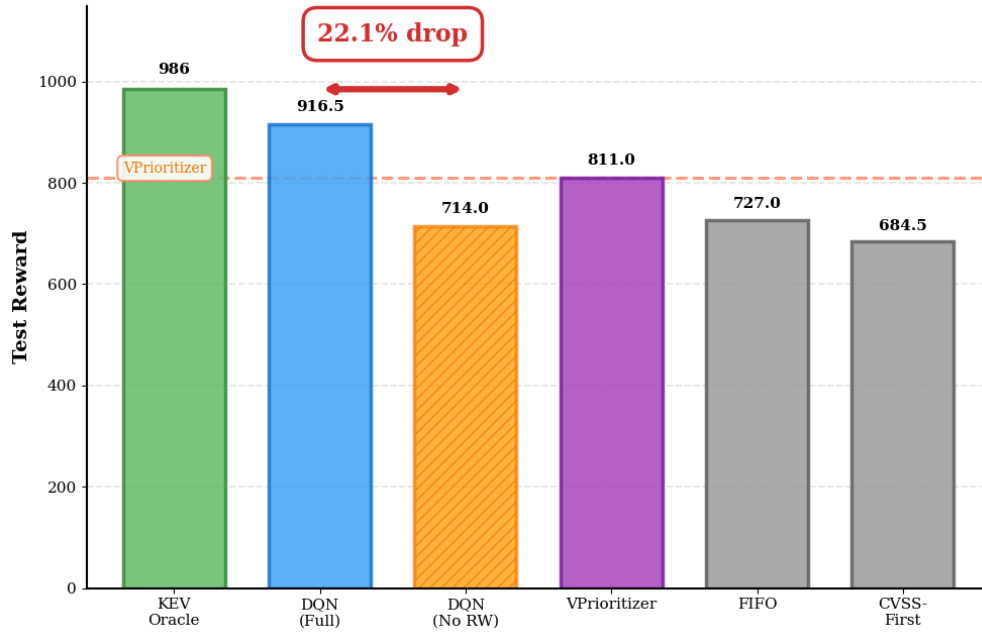


Figure 2: Performance comparison showing the impact of removing the ransomware flag from the DQN state space. The 15.5% performance degradation (845.50 to 714.00 reward) demonstrates that the model genuinely relies on this feature rather than compensating through CVSS and EPSS alone. Notably, the ablated model falls below VPrioritizer’s performance, indicating that ransomware campaign indicators provide critical context beyond static severity scores and exploitation probabilities.

Table 5: Action Distribution Comparison: Full Model vs. Ablated Model

Method	Monitor	30-day	7-day	Immediate
DQN (Full)	0.0%	7.0%	2.0%	91.0%
DQN (No Ransomware)	0.6%	3.1%	0.0%	96.4%
Difference	+0.6%	−3.9%	−2.0%	+5.4%

Note: The ablated model becomes more conservative, allocating 96.4% of resources to immediate patching compared to 91.0% for the full model. Medium-priority scheduling (30-day and 7-day windows) decreases substantially, suggesting the ransomware flag helps distinguish urgency levels within high-priority vulnerabilities.

7.1.2 Adaptive Capacity-Aware Decisions. Static heuristics prioritize independently without considering sequential dependencies. DQN learns temporal reasoning—balancing immediate exploitation with exploration.

7.1.3 Data-Driven Policy Refinement. VPrioritizer requires manual rule updates. DQN automates policy refinement through retraining on updated KEV data.

7.2 Practical Implications for Security Operations

7.2.1 Deployment in SOC. Our approach integrates with existing workflows:

- (1) Daily vulnerability scan → Extract features (CVSS, EPSS, ransomware)
- (2) DQN inference → Q-values for actions
- (3) Map to priority queues (Monitor / 30d / 7d / Immediate)
- (4) Quarterly retraining (15 min) on updated KEV data

7.2.2 Resource Optimization. The 4.3% improvement translates to faster remediation of exploited vulnerabilities and reduced waste on low-risk items.

7.2.3 Explainability and Trust. The action distribution analysis reveals interpretable patterns: 91% immediate patching aligns with 77% high-urgency composition.

7.3 Limitations and Threats to Validity

7.3.1 KEV Selection Bias. Training on KEV-confirmed vulnerabilities introduces selection bias toward already-exploited threats rather than exploitable-but-not-yet-exploited vulnerabilities. Because our evaluation is restricted to KEV-confirmed vulnerabilities, the results measure exploitation timing efficiency within the confirmed-threat subset rather than end-to-end enterprise triage performance.

In practice, operational patch queues contain a mix of KEV-confirmed and non-exploited vulnerabilities, where the latter typically outnumber the former by a large margin. Deploying the learned policy on such a mixed queue would require the agent to implicitly distinguish between vulnerabilities that will eventually appear in KEV and those that will not, a background filtering problem that the current formulation does not address. We expect the policy to continue to correctly prioritize ransomware-flagged and high-EPSS items, since these signals are present regardless of queue composition. However, capacity allocation decisions could be affected if the proportion of low-urgency items is large, as the agent was not trained to reason about their presence. Evaluating on mixed queues using a realistic enterprise dataset is an important next step toward operational deployment.

7.3.2 Training Data Scale. With 1,053 training examples, our dataset is modest by deep learning standards, and this scale imposes practical limits on what the agent can learn. The comparatively shallow network architecture (around 26,000 parameters) was partly selected to mitigate the risk of overfitting under this constraint, and the high test performance indicates that the agent generalizes reasonably well despite limited training data. However, several directions might remedy this limitation. First, because the KEV catalog grows by over 400 entries each year, simply extending the training window by a couple of years would yield around twice as many examples without any additional data-collection infrastructure. Second, synthetic trajectory generation - simulating realistic scheduling sequences from existing CVE feature distributions - could supplement the training set while maintaining the temporal ordering that avoids leakage. Third, transfer learning from related vulnerability datasets, such as the complete NVD corpus or organization-specific patch histories could provide a richer initialization before fine-tuning on KEV-confirmed examples. Each of these directions offers a practical path toward improving generalization without waiting for additional real-world exploitation evidence to accumulate.

7.3.3 Temporal Generalization Assumptions. Our evaluation assumes feature-urgency relationships remain stable over time. Major shifts in attacker behavior could degrade performance.

7.3.4 Feature Engineering and Reward Parameterization. Our ablation study shows that the presence of the ransomware flag is indeed meaningful, yielding a 15.5% reduction in reward when absent, though our state representation remains limited to six features. A typical organization has access to far richer contextual signals, such as asset criticality scores, network exposure metrics, and vendor threat intelligence. Testing additional ablations, such as CVSS-only or EPSS-only models would provide further insight into which features deliver the greatest marginal value for exploitation prediction.

The reward values were selected to reflect three operational realities. First, the penalty for missing a high-urgency vulnerability (-5) is larger in magnitude than the reward for correctly patching it ($+3$) because the cost of a successful ransomware exploitation far exceeds the cost of a delayed but eventually completed patch. Second, the asymmetry between HIGH and MED urgency penalties

(-5 vs. -3) reflects the difference between confirmed ransomware-targeted vulnerabilities and general high-severity items. Third, a $\pm 20\%$ sensitivity analysis suggests that relative rankings remain stable across plausible rewards parameterizations.

7.4 Comparison to Related Work

7.4.1 Vulnerability Prioritization. CVSS [9] has served as the default severity standard for years, but it assigns fixed scores at disclosure time and provides no signal about real-world exploitation a limitation documented empirically by Spring et al. [22]. EPSS [10] improved on this approach by estimating exploitation probability, yet it still treats vulnerabilities independently and ignores capacity constraints. Frameworks such as SSSVC [5] and VPrioritizer [21] combine multiple signals into structured decision rules, with VPrioritizer representing the strongest rule-based baseline in our evaluation. Our DQN agent improves the cumulative reward over VPrioritizer by 4.3% on unseen future vulnerabilities. A common limitation across these methods is that their decision boundaries remain fixed at design time and cannot adapt as attacker behavior changes [16]. Our work is the first to frame vulnerability prioritization as a capacity-constrained sequential decision problem and to learn a policy directly from confirmed evidence of exploitation.

7.4.2 RL for Cybersecurity. Prior RL work in cybersecurity focuses primarily on autonomous penetration testing [8, 18] and network defense [24], with recent work extending these approaches using large language models for predictive threat mitigation [25]. Amin et al. [1] remain the closest to our approach in applying DQN to vulnerability detection. These efforts address real-time offensive and defensive operations rather than the upstream planning problem of which known vulnerabilities to remediate before exploitation occurs. Our work is the first to apply RL to proactive vulnerability remediation using KEV as exploitation ground truth, under strict temporal evaluation conditions that reflect real deployment constraints.

7.5 Future Research Directions

Several directions emerge as natural extensions of this work. The most pressing of these is extending the state representation beyond the six features used here: adding asset-level context, such as criticality scores, network exposure metrics, and dependencies across systems, would enable an agent to distinguish between two vulnerabilities with identical CVSS and EPSS but very different impact on a given organization. A second is continuous learning, given the more than 400 annual additions to the KEV catalog. Allowing incremental policy updates driven by new confirmations would enable the agent to follow evolving attacker behavior without incurring the cost of complete retraining, thereby remedying the temporal generalization limitation discussed in Section 7.3. Finally, transferring learning across organizations provides an attractive route toward wider adoption: a meta-policy trained on the full KEV catalog could be specialized to a particular organization's patch history, alleviating data needs that until now have limited learning-based prioritization to the security teams of the data sources.

7.6 Broader Impact

This work is designed to support, not replace, the security analyst. Q-value outputs give practitioners a ranked list they can inspect and override based on organizational context. Two concerns deserve attention as the approach matures: the KEV catalog may not capture exploitation activity uniformly across all vulnerability types, so periodic evaluation is advisable, and publishing a model that reasons about exploitation carry some adversarial risk, though public inputs and 15-minute retraining reduce the practical impact of any attempt to game the system.

7.7 Malware Operators and Patch Timing Exploitation

Vulnerability prioritization systems are often evaluated primarily on technical remediation efficiency, without accounting for adversarial dynamics. However, malware operators actively exploit the temporal gap between vulnerability disclosure, KEV confirmation, and patch deployment. Understanding these adversarial patterns clarifies why reinforcement learning-based prioritization provides defensive advantages against exploitation-driven malware campaigns.

7.7.1 Exploitation Windows and Malware Deployment. Ransomware operators and Advanced Persistent Threat (APT) groups systematically monitor vulnerability disclosures, developing exploits for high-impact vulnerabilities within days of public disclosure [6]. This form of cyber-assault also accounted for most threat actors gaining initial access through unpatched vulnerabilities, and many incident analyses reported rapid time-to-exploit from the 2023 analysis of KEV-confirmed exploitation [6, 26]. Organizations that follow traditional CVSS-based prioritization often delay patching medium-severity vulnerabilities (CVSS 6.0–7.9) for 60–90 days, even though these vulnerabilities frequently appear in malware exploit chains when combined with other techniques.

7.7.2 Prioritization Errors and Malware Outbreak Scenarios. EPSS-First achieves only 78.71% KEV recall, leaving approximately 76 eventually-exploited vulnerabilities unaddressed during active exploitation windows. Our agent’s 100% KEV recall ensures all confirmed-threat items receive patch prioritization before malware operators weaponize them, directly closing the exposure window that enables initial access.

7.7.3 Capacity Constraints and Adversarial Advantage. Defender resource constraints work asymmetrically to favor malware operators. Defenders must secure all entry points simultaneously, while attackers need only one successful exploitation path. This asymmetry is captured in our formulation by the weekly patch capacity constraint ($k = 20$): organizations cannot patch all disclosed vulnerabilities at once and must make sequential decisions about resource allocation.

Static prioritization heuristics fail under this constraint because they evaluate vulnerabilities independently without considering temporal dependencies. CVSS-First exhausts capacity on high-severity items regardless of exploitation likelihood, while EPSS-First (as demonstrated in Section 6) depletes resources early in the week, leaving no capacity for ransomware-flagged items that arrive

later. Our learned policy discovers capacity-aware strategies that anticipate malware targeting patterns, allocating defensive resources to vulnerabilities most likely to enable ransomware deployment, botnet installation, or APT persistence.

8 Artifact Availability

To support reproducibility during review, we provide an anonymized artifact package through the conference submission system. The package contains several components designed to enable independent verification of our empirical results. An anonymized, view-only artifact is available at: <https://osf.io/rbuhv/>

First, we include the complete Gymnasium-compatible vulnerability remediation environment that implements the MDP formulation described in Section 3. This environment models patch capacity constraints, deployment delays, and progressive KEV revelation, enabling researchers to reproduce our experimental setup or extend it for related investigations.

Second, the package provides training scripts for the DQN agent and all six baseline methods evaluated in Section 6. These scripts include hyperparameter configurations matching those specified in Table 1, ensuring that others can replicate our training procedures exactly. We also include evaluation code that generates the results presented in Tables 3–4 and Figure 1.

Third, we also include the processed dataset derived from public sources (CISA KEV, EPSS API, and NVD), including the temporal splits used in our experiments. Providing the processed dataset avoids ambiguity introduced by preprocessing choices such as feature extraction, missing-value handling, and temporal alignment.

Fourth, the package includes trained model weights for both the full DQN agent (6 features) and the ablated variant (5 features, no ransomware flag) discussed in Section 6.7. These weights enable immediate evaluation without requiring 15-minute retraining, facilitating rapid experimentation and verification.

Finally, we provide comprehensive documentation with step-by-step instructions for environment setup, dependency installation, training execution, and results reproduction. The documentation includes troubleshooting guidance addressing common issues we encountered during development.

The complete codebase, trained model weights, and processed dataset are publicly available at the repository linked above to facilitate further research in reinforcement learning-based vulnerability prioritization.

9 Conclusion

Vulnerability remediation under resource constraints represents a critical operational challenge in modern cybersecurity. With over 40,000 CVEs disclosed annually and exploitation occurring within days, security teams require adaptive prioritization methods that focus limited capacity on vulnerabilities most likely to be exploited.

This paper introduced a reinforcement learning-based approach to vulnerability prioritization using CISA’s Known Exploited Vulnerabilities (KEV) catalog as exploitation ground truth. We formulated vulnerability remediation as a capacity-constrained sequential decision problem and demonstrated that Deep Q-Networks achieve a **4.3% improvement** over VPrioritizer, a recent rule-based method,

with **100% recall** on 357 unseen future vulnerabilities, closing **19.7% of the performance gap** to the oracle.

9.1 Key Contributions

- (1) **Novel Problem Formulation:** Capacity-constrained sequential decision problem with delayed exploitation revelation.
- (2) **Operational Simulation:** Gymnasium-compatible environment modeling real-world constraints.
- (3) **Empirical Validation:** 4.3% improvement over recent rule-based methods, 100% recall, strong generalization to future vulnerabilities.
- (4) **Ablation Evidence:** Feature removal experiments demonstrate the model learns genuine feature interactions rather than encoding simple heuristics (15.5% performance degradation when ransomware flag is removed).
- (5) **Practical Viability:** 15-minute training time, millisecond inference latency, compatible with existing SOC workflows.

9.2 Implications for Practice

Security operations centers can integrate RL-based prioritization with minimal disruption. The 4.3% improvement translates to faster remediation of vulnerabilities actively targeted by ransomware operators and malware campaigns, reducing the exposure window during which threat actors can achieve initial access. By maintaining 100% recall on KEV-confirmed vulnerabilities, the learned policy ensures that all eventually-exploited items receive appropriate prioritization before malware operators weaponize them, directly addressing the exploitation-driven compromise pattern consistent with widely reported exploitation-driven ransomware initial-access patterns.

9.3 Future Vision

As KEV continues to grow (400+ additions annually), the foundation for learning-based prioritization strengthens. We envision RL-based policies becoming standard practice in security operations, continuously learning from emerging threats to allocate defensive resources where they matter most.

Code, trained models, and processed datasets will be released upon publication to enable reproducibility and facilitate further research.

References

- [1] Rashid Amin, Muhammad Farhan, Faisal S. Alsubaei, and Eesa Alsolami. 2025. Cyber security enhancements with reinforcement learning: A zero-day vulnerability identification perspective. *PLOS ONE* 20, 5 (2025), e0324595.
- [2] Cybersecurity and Infrastructure Security Agency (CISA). 2021. Binding Operational Directive 22-01: Reducing the Significant Risk of Known Exploited Vulnerabilities. <https://www.cisa.gov/news-events/directives/bod-22-01-reducing-significant-risk-known-exploited-vulnerabilities>
- [3] Cybersecurity and Infrastructure Security Agency (CISA). 2022. CISA SSVC Guide. https://www.cisa.gov/sites/default/files/publications/SSVC_Guide_508.pdf
- [4] Cybersecurity and Infrastructure Security Agency (CISA). 2024. Known Exploited Vulnerabilities (KEV) Catalog. <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>
- [5] Cybersecurity and Infrastructure Security Agency (CISA). 2025. Stakeholder-Specific Vulnerability Categorization (SSVC). <https://www.cisa.gov/stakeholder-specific-vulnerability-categorization-ssvc>
- [6] Cybersecurity and Infrastructure Security Agency (CISA), Federal Bureau of Investigation (FBI), National Security Agency (NSA), and international partners. 2024. 2023 Top Routinely Exploited Vulnerabilities (AA24-317A). <https://www.cisa.gov/news-events/cybersecurity-advisories/aa24-317a>
- [7] Farama Foundation. 2024. Gymnasium: A Standard Interface for Reinforcement Learning Environments. arXiv:2407.17032 <https://arxiv.org/abs/2407.17032>
- [8] José Álvaro Fernández Carrasco, Iñigo Amonariz Pagola, Raúl Orduna Urrutia, and Rodrigo Román. 2024. CYBERSHIELD: A Competitive Simulation Environment for Training AI in Cybersecurity. In *2024 11th International Conference on Internet of Things: Systems, Management and Security (IOTSMS)*. IEEE, Piscataway, NJ, 11–18. doi:10.1109/IOTSMS62296.2024.10710208
- [9] FIRST. 2019. Common Vulnerability Scoring System v3.1: Specification Document. <https://www.first.org/cvss/v3.1/specification-document>
- [10] Jay Jacobs, Sasha Romanosky, Benjamin Edwards, Michael Roytman, Armin Sarabi, and Idris Adjerid. 2021. Exploit Prediction Scoring System (EPSS). *Digital Threats: Research and Practice* 2, 3 (2021), 1–17. doi:10.1145/3436242
- [11] Yuning Jiang, Nay Oo, Qiaoran Meng, Hoon Wei Lim, and Biplab Sikdar. 2025. A Survey on Vulnerability Prioritization: Taxonomy, Metrics, and Research Challenges. arXiv:2502.11070 [cs.CR] <https://arxiv.org/abs/2502.11070>
- [12] Diederik P. Kingma and Jimmy Ba. 2014. Adam: A Method for Stochastic Optimization. arXiv:1412.6980 <https://arxiv.org/abs/1412.6980>
- [13] Éireann Leverett. 2025. The 2024 Vulnerability Forecast: Year in Review. <https://www.first.org/blog/20250106-Vulnerability-Forecast-Year-in-Review-Reports> 40,704 CVEs published in calendar year 2024.
- [14] Éireann Leverett. 2025. Vulnerability Forecast for 2025. <https://www.first.org/blog/20250607-Vulnerability-Forecast-for-2025> Forecasts 45,505 $\pm$ 4,363 CVEs for 2025.
- [15] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fiedelnd, Georg Ostrovski, et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529–533. doi:10.1038/nature14236
- [16] Mohammed Moustaid, Soufiane Hamida, Abdelaziz Daaif, and Bouhaib Cherradi. 2025. Toward Dynamic Risk Assessment: Machine Learning and LLMs in Software Vulnerability Prioritization. In *2025 12th International Conference on Wireless Networks and Mobile Communications (WINCOM)*. IEEE, Piscataway, NJ, 1–6.
- [17] National Institute of Standards and Technology (NIST). 2024. National Vulnerability Database. <https://nvd.nist.gov/>
- [18] Huynh Phuong Thanh Nguyen, Zhi Chen, Kento Hasegawa, Kazuhide Fukushima, and Razvan Beuran. 2024. PenGym: Pentesting Training Framework for Reinforcement Learning Agents. In *Proceedings of the 10th International Conference on Information Systems Security and Privacy (ICISSP 2024)*. INSTICC, SciTePress, Setubal, Portugal, 498–509. doi:10.5220/0012367300003648
- [19] Adam Paszke et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*. Curran Associates, Vancouver, Canada, 8024–8035.
- [20] Bonan Ruan, Jiahao Liu, Weibo Zhao, and Zhenkai Liang. 2024. VulZoo: A Comprehensive Vulnerability Intelligence Dataset. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24), Tool Demonstrations*. ACM, New York, NY, 2334–2337. doi:10.1145/3691620.3695345
- [21] Naoyuki Shimizu and Masaki Hashimoto. 2026. Vulnerability management chaining: An integrated framework for efficient cybersecurity risk prioritization. *IEEE Access* 14 (2026), 1–15. doi:10.1109/ACCESS.2025.11397608
- [22] Jonathan Spring, Eric Hatleback, Allen Householder, Art Manion, and Deana Shick. 2021. Time to Change the CVSS? *IEEE Security & Privacy* 19, 2 (2021), 74–78.
- [23] Tenable. 2025. Enhancements to Tenable Vulnerability Priority Rating (VPR). <https://www.tenable.com/whitepapers/enhancements-to-tenable-vulnerability-priority-rating-vpr>
- [24] TTCP CAGE. 2023. CAGE Challenge 4: Autonomous Cyber Operations.
- [25] MP Varsha, S Samyuktha, S Manochithra, and K Sudha. 2025. LLM-Powered Deep Reinforcement Learning for Predictive Cyber Threat Mitigation. In *2025 5th International Conference on Ubiquitous Computing and Intelligent Information Systems (ICUIS)*. IEEE, Piscataway, NJ, 1020–1025.
- [26] Verizon. 2024. 2024 Data Breach Investigations Report. <https://www.verizon.com/business/resources/reports/dbir/>